When Does Keeping Agent Sessions in GPU Memory Pay Off?

A Rust Serving Engine and a Replay Study on Real Agent Traces

Nahuel Alejandro Nucera
nahuelnucera990@gmail.com

November 2026

Abstract

AI agents call a language model, run a tool, and call the model again with the whole history attached. Keeping an agent's KV cache in GPU memory while it waits on a tool avoids recomputing that history, but holds memory other requests could use. Prior systems (InferCept, Pensieve, Continuum, TokenCake and others) model this trade-off and evaluate it against earlier versions of vLLM. We ask how it plays out against the current prefix caches of vLLM 0.33.0 and SGLang 0.5.23. We built a serving engine in Rust (about 1,700 lines) with per-session KV retention, cross-session prefix sharing and session-level admission, and replayed 24 real coding-agent sessions (791 turns) against all three engines with the same GPU, model and KV cache size, two runs per configuration. With a fixed 0.5-second tool wait, close to the median gap measured for coding agents, our engine finishes the workload 1.31x faster than the best baseline (1.45x faster than vLLM) and returns the first token about 5x sooner. With a 5-second wait the gain is 1.13x (1.24x over vLLM). With a 60-second wait, all engines finish within 2% of each other and our engine is slower to the first token, consistent with prior reports that retention stops paying off when agents are away long. Three pinning policies we tried, each with a pass/fail criterion written down before seeing its results, did not beat the baselines at 60 seconds.

1 Introduction

A coding agent works in turns. It sends the conversation so far to the model, receives a tool call, runs the tool (a shell command, a file edit), appends the result and sends the conversation again. In the traces we use, the median session has about 60 turns and grows to tens of thousands of tokens. Every turn re-sends everything before it.

Serving engines reuse repeated prefixes. vLLM stores the KV cache in fixed-size blocks [1] and, in current versions, reuses blocks identified by a hash of their tokens; SGLang keeps a radix tree of cached prefixes [2]. Both evict least recently used entries when memory runs out (vLLM per its source code). An agent waiting on a tool looks idle to such a cache, so its history is a natural eviction candidate and must be recomputed when the agent returns.

Whether to keep or drop that history is a known trade-off. InferCept [6] compares the memory a paused request holds, times how long it waits, against the cost of recomputing it. Pensieve [7] keeps conversation history across turns and reports that its advantage shrinks as the time between turns grows. Continuum [4] pins an agent's KV cache across tool calls with a time-to-live chosen from each tool's past durations, and several systems since then refine the policy (Section 2). These systems were evaluated against earlier versions of vLLM and SGLang, mostly on traces whose tool calls last one to two seconds.

We ask a narrower question: against the prefix caches of current engines (vLLM 0.33.0, SGLang 0.5.23), on real agent traces and with matched memory, how much does keeping sessions in GPU

memory still help, and when does it stop? To answer it we wrote a serving engine with sessions as a first-class concept and compared it with both engines under controlled conditions.

Contributions:

- An open-source serving engine in Rust (`https://github.com/Nahuel990/agent-session-engine`) with per-session KV retention, cross-session prefix sharing, session-level admission and three pinning policies, checked against vLLM and an FP32 reference (Sections 3, 4). Its mechanisms follow prior work; we do not claim them as new.
- A replay benchmark of 24 real agent sessions with matched KV cache size across engines, run inside the server container to avoid network effects, with repeat runs and pass/fail criteria written down before seeing results (Section 5).
- Measurements against current engines: an end-to-end gain over the best baseline of 1.31x with 0.5-second tool waits and 1.13x with 5-second waits, about 5x faster time to first token at both, and parity with 60-second waits (Section 6).
- Negative results for three pinning policies, consistent with reports that more retention does not necessarily improve serving [11, 14] (Section 7).

2 Background and related work

Prefix caching in current engines. vLLM stores the KV cache in fixed-size blocks [1]; the paper shares blocks for predefined prefixes through reference counts and copy-on-write. Version 0.33.0 goes further: it identifies a block by a hash chained over its tokens and the previous block, reuses matching blocks across requests, and evicts unreferenced blocks in least-recently-used order (we checked its source code). Its OpenAI endpoint accepts a `session_id`, but in this version it only labels KV-cache events; open proposals to add retention hints to vLLM and SGLang were not merged when we checked. SGLang keeps cached prefixes in a radix tree and evicts the least recently used leaf first [2]. Version 0.5.23 schedules first-come-first-served by default (the paper's longest-prefix-match policy is available as `lpm`) and offers several eviction policies, including Tail-Optimized LRU [3], which first evicts each conversation's cached tail beyond what keeps its next turn's uncached prefill under a threshold, then falls back to LRU. SGLang 0.5.23 also has a session API on its native `/generate` endpoint; its OpenAI chat endpoint forwards only a `session_id`.

Keeping KV across tool calls. InferCept [6] chooses, per paused request, to preserve, discard or swap its KV by comparing the memory it would hold over the pause with the cost of recomputing it, and notes that no single strategy is universally applicable. Continuum [4] pins KV with a time-to-live chosen from the reload cost, the expected queueing delay and the learned distribution of each tool's duration, and orders waiting requests by their program's arrival time; its evaluation traces have mean tool times of 0.9 and 1.9 seconds. TokenCake [9] offloads idle KV during function calls using per-function duration forecasts and a benefit gate; Astraea [10] picks preserve, discard or swap by memory pressure with per-API-category latency estimates. Choi and Joshi [11] price retention as GPU opportunity cost for waits of minutes to hours and report that enforced residency reduced goodput by 41%; MORI [12] moves the most idle programs out of GPU memory and calls per-call duration prediction unreliable, as does a study that has tools report their own progress [13]. A simulator of agent serving [14] finds that time-to-live retention under first-come-first-served scheduling changes job completion time by +0.4%. Norgren [5] keeps a persistent KV cache per session with LRU eviction of conversation tails and reports 2.1 to 4.2x lower per-turn

latency than vLLM and SGLang on short synthetic workflows on one L40S, from processing only new tokens plus speculative decoding.

Multi-turn conversation caching. Pensieve [7] keeps conversation KV in GPU and CPU memory and evicts by recompute cost per unit of idle time; with longer user think times its throughput falls and its gap to vLLM narrows. CachedAttention [8] saves session KV to host memory and disk. EfficientAgent [15] argues that while one agent waits for its tool, the server processes all other agents' contexts, so reuse depends on the reusable context of the whole agent pool fitting in memory. A production trace study [16] measures reuse times; RR-Evict [17] and Fan et al. [18] propose eviction policies for idle agents in SGLang that outperform LRU.

Session admission. Autellix [19] schedules agent programs by the service they have received. ThunderAgent [20] pauses programs when it detects thrashing. CacheWise [21] admits resident sessions first and reports higher tail latency because it defers new requests. Two studies of prefix-cache collapse under load [22, 23] show that a session cap or admission budget preserves reuse.

How long tools take. Measured coding-agent tool times are short with a heavy tail: median gaps of 0.42 s and 48 ms on Terminal-Bench and SWE-bench Pro [22], and mean Bash times of 1.3 to 2.6 s for Claude Code on coding benchmarks, with some tools running for minutes [24].

Our position. Our retention, pinning and admission mechanisms follow this line of work: the adaptive policy follows Continuum's per-tool learned time-to-live, the capped policy is a simple instance of InferCept's keep-or-recompute comparison, and our admission rule resembles CacheWise's. What we add is a controlled measurement against the current versions of vLLM and SGLang on real agent traces, with matched memory and repeat runs, an open implementation, and the policies that did not work.

3 Design

Sessions. A request may carry a `session_id`. When the request finishes, the engine keeps the session's token sequence and KV blocks. The next request of the same session reuses the longest common token prefix and computes only what follows.

Prefix sharing. Each full 64-token block gets a hash chained from the previous block's hash, so a hash identifies the whole prefix up to the end of its block. An index maps hashes to physical blocks. A new request takes the longest run of indexed blocks matching its tokens, from any session. Blocks are reference counted; a block nobody holds moves to an LRU list but stays in the index until its space is handed out. Indexed or shared blocks are never written: if a session must roll back into the middle of one, the rollback point is moved down to the block boundary and the rest is recomputed. This matters for agents: in our traces all 24 sessions start with the same 5,636 tokens (system prompt and tool definitions). Without sharing, those would be stored 24 times, about 78% of the KV cache.

Pinning. After a turn, a session's KV may be *pinned*: kept out of eviction for a while so that it survives the agent's tool call. We built three policies:

- **Fixed**: pin for 30 seconds after every turn.

- **Adaptive** (following Continuum's per-tool learned time-to-live [4]): the engine learns, per tool name, the gap between a turn that called the tool and the session's next request, and pins for the 90th percentile of that gap times 1.5 plus 2 seconds. A turn the model ends without a tool call is not pinned.
- **Capped** (a simple instance of InferCept's keep-or-recompute comparison [6]): pin only if the learned wait is no longer than the time needed to recompute a full KV cache (capacity divided by prefill speed, 173,200 tokens / 10,000 tokens per second, about 17 seconds), and only while pinned idle sessions hold at most half the cache.

Admission. A turn of a session already in memory is admitted first. A new session is admitted only if every resident session, counted at its current length times 1.2, plus the new prompt, still fits in the cache. Otherwise it waits. If running requests cannot grow, the most recently admitted one is preempted: its blocks are freed and it is recomputed later.

Steps. Each step processes at most 4,096 new tokens: running requests that generate one token go first, the rest of the budget goes to prompts in chunks. Steps in which every request generates exactly one token use a dedicated decode kernel.

4 Implementation

The engine is 1,709 lines of Rust. Model code (Qwen3) uses candle [25] (a fork at revision 8ac335c) and attention-rs [26] (revision 3c258f0) for paged attention through FlashInfer [27]. Q, K and V projections, and the gate and up projections, run as single stacked matrix multiplications; the residual add and RMSNorm after attention, and SiLU with multiplication, use fused kernels. The server speaks the OpenAI chat-completions API with streaming, renders the model's own chat template, and parses tool calls. Builds and benchmarks run on Modal; the engine runs on one NVIDIA L40S.

Correctness. We checked the engine in four ways. (1) Teacher-forced against vLLM on the same 64 tokens of Qwen3-8B: the engine's top token matches at all 64 steps. (2) Against an FP32 run of the same model in Hugging Face Transformers: mean absolute log-probability error 0.0056 on the reference tokens (vLLM: 0.0043), and the same top token as FP32 at all 64 steps. (3) A session's second turn on a live cache gives the same greedy output as a fresh run over the full history. (4) A request that reuses another request's cached prefix gives the same greedy output as on an empty cache.

5 Methodology

Workload. We use real coding-agent trajectories from SWE-rebench-OpenHands [28] (67,074 sessions collected with OpenHands and Qwen3-Coder-480B; CC-BY-4.0). We sampled 64 sessions with a fixed seed and use the first 24. Each session is replayed turn by turn: the prompt for turn t is every recorded message before the t-th assistant message, with the recorded tool definitions. The engine generates 256 tokens per turn (end-of-sequence ignored, so all engines do the same work). Between turns the client waits a fixed time standing in for the tool call: 0.5, 5 or 60 seconds. The 0.5-second wait is close to the median gaps measured for coding agents [22]; 5 and 60 seconds stand for slower tools and long waits. A session stops after its prompt passes 30,000 tokens. All 24

Table 1: Wall time for all 24 sessions, two runs each (seconds; lower is better). Ratios use means.

Engine	Tool wait 0.5 s	Tool wait 5 s	Tool wait 60 s
vLLM 0.33.0	2264 / 2222	2342 / 2333	4106 / 4128
SGLang 0.5.23, LRU	2013 / 2048	2179 / 2154	4069 / 4154
SGLang 0.5.23, TLRU	2067 / 2024	2121 / 2152	4078 / 4093
This engine (capped)	**1506 / 1586**	**1868 / 1908**	4083 / 4011
vs best baseline	1.31x (LRU)	1.13x (TLRU)	1.01x (TLRU)
vs vLLM	1.45x	1.24x	1.02x

sessions run concurrently. Our engine receives a `session_id` per session; vLLM and SGLang rely on their prefix caches.

Setup. Qwen3-8B (revision b968826), bfloat16, one L40S per engine. Every engine gets the same KV cache size: vLLM 0.33.0 allocates 173,200 tokens at `gpu_memory_utilization` 0.92; we set our engine to 2,706 blocks of 64 tokens (173,184) and SGLang 0.5.23 to `--max-total-tokens 173200 --mem-fraction-static 0.92` (with its default memory fraction it allocated only 158,237). SGLang runs with LRU eviction and, separately, with TLRU: threshold 4,096 tokens (the example value in SGLang's documentation, about 0.35 s of prefill at our measured prompt-processing speed) and next-prompt estimate 434 tokens (the median per-turn prompt growth on these traces). SGLang's documentation and the TLRU paper suggest deriving the threshold from a time-to-first-token target and using the mean prompt growth (735 tokens here), so our TLRU configuration is not tuned as they recommend. The replay client runs inside the server's container and connects to localhost: through Modal's proxy, streamed chunks arrived in bursts for every engine, which made time-to-first-token measurements unusable.

Metrics. *Wall time*: time to finish all 24 sessions (primary). *Recomputed history*: tokens the engine had already seen on the previous turn of the same session but did not serve from cache, as a share of all prompt tokens. *Time to first token* (TTFT): from sending a turn to receiving its first streamed token, median and 90th percentile over all turns.

Repeats and criteria. Every configuration ran twice. Identical runs of our engine differed by up to 7.7% in wall time; vLLM's by 0.5%. Before seeing the results of each memory-policy change we wrote down, with a timestamp, the result that would count as a pass and the decision we would take otherwise.

6 Results

Realistic tool calls. With 0.5-second tool calls, our engine finishes in 1506 and 1586 seconds, against 2013–2067 for SGLang and 2222–2264 for vLLM: 1.31x faster than the best baseline (SGLang LRU) and 1.45x faster than vLLM by mean wall time. Both of our runs beat every baseline run, which was the criterion we wrote down before running. The baselines recompute 51–55% of prompt tokens as history; our engine recomputes 7.5% and 15.6% in its two runs. The median turn gets its first token in 4.5–6.8 seconds versus 28.6–36.3 seconds. At this wait SGLang is about 10% faster than vLLM, and TLRU again changes SGLang's results by about 1%.

Table 2: Time to first token, median and 90th percentile over all turns, per run (seconds), and recomputed history (% of prompt tokens).

Engine	TTFT p50	TTFT p90	Recomputed
Tool wait 0.5 s			
vLLM	28.6, 28.8	50.2, 48.0	55.1, 54.9
SGLang LRU	35.0, 35.9	51.0, 51.5	50.6, 51.8
SGLang TLRU	36.3, 36.0	53.4, 50.6	51.7, 52.5
This engine	**4.5, 6.8**	**9.9, 25.5**	**7.5, 15.6**
Tool wait 5 s			
vLLM	26.8, 24.9	43.2, 43.6	55.5, 54.9
SGLang LRU	29.2, 29.4	44.0, 46.8	53.5, 52.7
SGLang TLRU	29.3, 25.3	45.8, 44.9	53.0, 52.7
This engine	**5.1, 6.3**	**26.0, 28.8**	**18.6, 23.1**
Tool wait 60 s			
vLLM	3.5, 3.8	16.6, 17.0	55.0, 54.9
SGLang LRU	6.1, 7.1	18.4, 19.4	52.6, 51.8
SGLang TLRU	6.3, 6.4	18.1, 18.8	53.1, 52.9
This engine	13.8, 11.5	23.0, 21.1	56.5, 54.6

Short tool calls. With 5-second tool calls (Table 1), our engine finishes 1.13x faster than the best baseline and 1.24x faster than vLLM, in both runs. The baselines recompute more than half of all prompt tokens as history they had already seen; our engine recomputes 19–23%. The median turn gets its first token about 5x sooner (Table 2). Among the baselines, SGLang is 8–9% faster than vLLM, and TLRU changes SGLang's results by about 1%.

Long tool calls. With 60-second tool calls, all engines finish within 2% of each other, within the run-to-run spread of our engine. Our engine recomputes about as much history as the baselines, and its first token is slower: median 11.5–13.8 seconds versus 3.5–3.8 for vLLM and 6.1–7.1 for SGLang. With long waits, few agents need the GPU at the same moment, so queues are short for every engine. Our likely explanation for the slower first token, which we did not measure separately, is that our admission rule makes sessions wait where the baselines' first-come-first-served queues do not; CacheWise reports the same effect of deferring new requests [21]. The 60-second result is consistent with prior reports that retention stops paying off when the gap between turns grows [7, 11, 12].

Prefix sharing. Adding cross-session prefix sharing to the capped policy moved our engine at 60 seconds from 3–4% slower than vLLM to parity, and at 5 seconds from 2097 to 1888 seconds mean wall time (Section 7).

Raw speed. Without any reuse, our engine generates tokens 15–21% slower than vLLM (25.1 / 27.8 / 29.7 ms per token at 1 / 8 / 24 concurrent requests, versus 21.8 / 23.5 / 24.6 ms; SGLang 21.5 / 24.5 / 25.3 ms). It processes a single long prompt at about vLLM's speed (11,862 versus 12,419 tokens per second for 3,000 tokens) but does not speed up when several prompts arrive together (11,791 versus 14,808 for eight). A profile of prompt processing showed RMSNorm costing

Table 3: Memory policies tried (mean wall time in seconds, two runs each; vLLM: 2338 at 5 s, 4117 at 60 s).

Policy	Commit	5 s	60 s
Fixed 30 s pin	de1aff9	2060	5342
Adaptive pin per tool	de1aff9	not run	6575
Capped pin	5cd8f72	2097	4261
Capped pin + prefix sharing	754089d	2863	stopped
Capped pin + prefix sharing, fixed	3a64258	**1888**	**4047**

more GPU time per token than all matrix multiplications before we switched to a fused kernel. Our wins come from doing less work, not from faster kernels.

7 What did not work

Table 3 lists every memory policy we measured.

Fixed pin. At 60 seconds, a 30-second pin expires before the agent returns. The session then counts as new, waits behind the admission rule, and its trimmed cache must be recomputed: 85% of history recomputed, 1.3x slower than vLLM.

Adaptive pin. Learning the wait per tool fixed the expiry problem (53% recomputed, about vLLM's level) but made wall time worse: 1.6x slower than vLLM and a 90th-percentile TTFT of over four minutes. By the policy's formula, a 60-second wait leads to pins of roughly 90 seconds; our reading, not measured directly, is that these idle pinned sessions blocked new ones and left the GPU with too little work. Our pre-registered criterion counted this policy as a pass because we had written "wall time *or* recomputed history"; we report it as worse on the metric that matters, and since then name one primary metric per experiment.

Capped pin. Pinning only when waiting is cheaper than recomputing, and capping pinned memory, came within 3–4% of vLLM at 60 seconds but still failed the criterion.

Prefix sharing, first version. Our first integration released whole sessions to the shared cache under pressure; a session without blocks then counted as new and waited behind the admission rule, giving a 90th-percentile TTFT above 100 seconds and failing at 5 seconds. We declared a single fix in advance (a returning session skips the new-session gate even if its blocks were released), kept the criterion unchanged, and reran once (last row).

Fused RMSNorm. The fused RMSNorm kernel from attention-rs brought prompt processing to vLLM's speed for single prompts but moved our log-probabilities further from FP32 (mean error 0.0071 versus 0.0053) and changed the top token at one of 64 steps. We use candle's fused kernel instead (error 0.0056, no top-token change, 11–14% faster prefill than unfused).

Measurement pitfalls. Two harness problems produced misleading numbers before we caught them: time to first token measured through a proxy (chunks delivered in bursts) and HTTP

connections reused across a 5-second tool wait, which raced the server's 5-second keep-alive timeout and failed requests for vLLM and our engine alike.

8 Limitations

Tool waits. Our tool waits are fixed. Measured coding-agent tool calls have median gaps under half a second and means of one to two seconds, with a heavy tail [22, 24, 4]. Our 0.5-second wait is close to the median, but we did not test mixed or heavy-tailed distributions, where short and long waits arrive together.

Baselines. We compare against the default GPU-only prefix caches of vLLM and SGLang. We did not test CPU offload tiers (LMCache, SGLang HiCache), newer eviction policies for idle agents [17, 18], or SGLang's streaming sessions, and our TLRU configuration does not follow the tuning its authors recommend. Stronger baselines may narrow or remove our 5-second gain.

Scope. One model (Qwen3-8B), one GPU type (L40S), one load level (24 concurrent sessions). Replayed turns ignore the model's actual output: the next turn is always the recorded one. vLLM and SGLang reject prompts over 32,768 tokens and our engine does not, so it served 2–3 more turns of one session than the baselines. Two runs per configuration bound but do not characterize the variance of our engine, whose runs differed by up to 7.7%.

9 Conclusion

Against the current prefix caches of vLLM 0.33.0 and SGLang 0.5.23, keeping agent sessions in GPU memory still helps when agents come back within a few seconds: on real coding-agent traces, 1.31x faster end to end than the best baseline with 0.5-second tool calls and 1.13x with 5-second calls, and about 5x faster to first token at both. When agents are away for a minute the advantage disappears, as prior work on retention trade-offs predicts, and none of our three pinning policies beat the established caches; sharing common prefixes across sessions was what brought our engine back to parity. Testing heavy-tailed tool times and stronger baselines with CPU offload are the next steps. The code, data and the scripts that produce every number in this paper are available under the MIT license at `https://github.com/Nahuel990/agent-session-engine`.

References

[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180.

[2] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, Y. Sheng. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104, 2023.

[3] W. Zhang, Y. Li, C. C. Moallemi, T. Peng. Tail-Optimized Caching for LLM Inference. arXiv:2510.15152, 2025.

[4] H. Li, R. He, Q. Mang, Q. Zhang, H. Mao, X. Chen, H. Zhou, H. Zhang, A. Cheung, J. Gonzalez, I. Stoica. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230.

[5] V. Norgren. Stateful Inference for Low-Latency Multi-Agent Tool Calling. arXiv:2605.26289, 2026.

[6] R. Abhyankar, Z. He, V. Srivatsa, H. Zhang, Y. Zhang. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. arXiv:2402.01869, 2024.

[7] L. Yu, J. Lin, J. Li. Stateful Large Language Model Serving with Pensieve. EuroSys 2025. arXiv:2312.05516.

[8] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, et al. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. USENIX ATC 2024. arXiv:2403.19708.

[9] Z. Bian, F. Wu, Z. Li, T. Ma, Y. Zhuo. TokenCake: A KV-Cache-centric Serving Framework for LLM-based Multi-Agent Applications. arXiv:2510.18586, 2025.

[10] H. Ni, J. Zhang, G. Li, Z. Wang, R. Wu, C. Zhang, et al. Astraea: A State-Aware Scheduling Engine for LLM-Powered Agents. arXiv:2512.14142, 2025.

[11] M. Choi, A. Joshi. Adaptive KV Retention for LLM Agents at Human-Approval Timescales. arXiv:2608.30830, 2026.

[12] T. Xia, H. Li, Z. Li, X. Chen, H. Kang, Y. Qiao, et al. Idleness is Relative: Exploiting Tool-Call Idle Windows for Offloading in Agentic Systems with MORI. arXiv:2606.00866, 2026.

[13] Y. Liu, Y. Zhang, F. Li, H. Zhang. Ask the Tool, Don't Guess: Agent Tool Calls Hold Their Progress, and the Serving System Should Read It. arXiv:2609.18849, 2026.

[14] AgentServeSim: Serving-System Simulation and Policy Search for LLM Agent Programs. arXiv:2606.09613, 2026.

[15] K. Shao, J. Chen, J. Yu, X.-H. Li, C. Tao, Y. Wang, et al. EfficientAgent: What Makes KV Cache Offloading Work for Concurrent Agents? arXiv:2609.33762, 2026.

[16] J. Wang, J. Han, X. Wei, S. Shen, D. Zhang, C. Fang, et al. KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider. USENIX ATC 2025. arXiv:2506.02634.

[17] Z. Pan, C. Wu, Z. Hu, X. Qiang, Z. Yu, Y. Ding. RR-Evict: Fine-Grained Prefix Cache Eviction beyond LRU for Agentic LLM Serving. arXiv:2609.32278, 2026.

[18] J. Fan, H. Meng, Y. Xia, H. M. Chen, D. Li, D. S. Nikolopoulos. Rethinking KV Cache Eviction for Agentic LLM Serving. arXiv:2611.00391, 2026.

[19] M. Luo, X. Shi, C. Cai, T. Zhang, J. Wong, Y. Wang, et al. Autellix: An Efficient Serving Engine for LLM Agents as General Programs. arXiv:2502.13965, 2025.

[20] H. Kang, Z. Li, W. Xu, X. Yang, Y. Chen, J. Wang, et al. ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System. arXiv:2602.13692, 2026.

[21] S. Tiwari, T. Chugh, N. Rickert, S. Peter, R. Mahajan, H. Shen. CacheWise: Understanding Workloads and Optimizing KVCache Management for Efficiently Serving LLM Coding Agents. arXiv:2606.16824, 2026.

[22] C. Lu, B. Zhou, Y. Lin, D. Ustiugov. When Cache Pressure Becomes a Trap: Metastability in Agent Serving. arXiv:2611.02543, 2026.

[23] A. Valse, A. Jannesari. Inside the Trap: A Renewal Law for Prefix-Cache Collapse in LLM Agent Serving. arXiv:2611.08902, 2026.

[24] Y. Yuan, A. Nayak, S. Kundu, N. Talati. Agentic AI Workload Characteristics. arXiv:2605.26297, 2026.

[25] Hugging Face. candle: a minimalist ML framework for Rust. https://github.com/huggingface/candle; fork used: https://github.com/guoqingbao/candle.

[26] G. Bao. attention.rs. https://github.com/guoqingbao/attention.rs.

[27] FlashInfer: Kernel Library for LLM Serving. https://github.com/flashinfer-ai/flashinfer.

[28] Nebius. SWE-rebench-OpenHands-Trajectories. https://huggingface.co/datasets/nebius/SWE-rebench-openhands-trajectories.